

이동형

DevOps Engineer | 경력기술서

GitHub: github.com/somaz94 Blog(EN): somaz.blog Blog(KR): somaz.tistory.com Portfolio: somaz.blog/portfolio

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- AWS EKS·GCP GKE 프로덕션 운영·비용 최적화 (하이브리드 전환 20% 절감, GCP 마이그레이션 50% 절감 — Fargate → GKE 노드 풀 재배포 + MSP 할인 확보, 절감분 신규 프로젝트 재투자)
- 신규 외부 퍼블리싱 게임의 그린필드 EKS 프로덕션 환경 단독 구축·런칭 — Karpenter로 CI 빌드는 ARM 다중 family 스팟, 게임 워크로드는 on-demand로 분리한 2개 NodePool 운영, IRSA / Pod Identity로 정적 키 없는 인증, 플랫폼 컴포넌트는 100% GitOps로 선언적 관리, 2026.07 소프트웨어 런칭 개시
- Terraform IaC 인프라 자동화·버전 관리
- AI 워크로드용 GPU 인프라 구축·운영 — GCP A100 2장 학습 서버는 Terraform IaC로 구축(너디스타), 온프레미스 GPU 서버에는 Ollama·Open WebUI 기반 사내 LLM 추론 서비스를 Kubernetes로 운영(팬텀)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- GitHub Actions·GitLab CI·Jenkins·ArgoCD 기반 CI/CD 파이프라인 설계·고도화
- 배포 시간 67% 단축, GitOps 지속 배포 전환으로 시장 대응 속도 향상, 롤백 시간 95% 감소 (30분 → 1~2분)
- 프로덕션 게임 서비스를 Blue-Green → Argo Rollouts Canary로 전환 — 승격 시 healthy target이 순간 0이 되며 발생하던 503 레이스 제거, 하위 호환 패치는 무중단 배포

모니터링 및 오피서빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- ELK → EFK Stack 전환 후 ECK Operator 기반 Elastic Stack 9.0 CRD 선언형 관리로 재설계 (TLS 자동 회전, 롤링 업그레이드 자동화)
- kube-prometheus-stack 기반 커스텀 알람 규칙·Grafana 대시보드·DB exporter (MySQL, Redis, Elasticsearch, PostgreSQL) 통합
- OpenTelemetry + Tempo로 분산 추적을 더해 Metrics ↔ Traces ↔ Logs 3-signal 관측성 확보 — RED 메트릭은 샘플링 이전 100% span에서 생성해 정확도를 지키고, trace 저장 단계에만 tail sampling을 적용해 저장 비용 절감

자동화 및 팀 역량 강화

Python·Bash 운영 자동화 도구 구축과, 고객사 운영팀이 스스로 운영하도록 만드는 교육·자료 표준화

- Shell → Python(표준 라이브러리만) 마이그레이션 + 컴포넌트 단계별(wave) 순차 자동 배포·업그레이드 MR 자동 생성
- DP사 운영팀이 Kubernetes를 스스로 운영하도록 정착 — 6개월 교육 프로그램 설계·운영 (수강자 평균 10명·회당 4시간), 교육 자료를 표준화해 타 고객사 교육에도 재활용
- 업무 밖에서도 같은 방식으로 — 공개 API 10여 곳에서 데이터를 수집 → 품질 검증 → 재배포하는 파이프라인을 직접 만들어 평일 하루 2회 스케줄로 자동 실행(경제 대시보드), 백엔드 없는 [다이아그램 에디터](#)도 함께 공개 운영

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크·로깅 스택의 전환, 재사용 가능한 공용 차트 오픈소스 공개, 그리고 업스트림 기여

- Ingress-nginx 다수 인스턴스 → NGINX Gateway Fabric 단일 컨트롤플레인 + 클래스별 Gateway CR 마이그레이션 (전체 cutover, wildcard TLS 통합)
- 오픈소스 Helm 차트(nginx-gateway-cr, elasticsearch-eck, kibana-eck)를 비롯해 Kubernetes CRD 컨트롤러·composite GitHub Actions·Ansible 컬렉션 다수 공개, ArtifactHub / GitHub Marketplace / Ansible Galaxy 등록
- 필요한 기능이 없으면 업스트림에 직접 기여 — apache/airflow · OpenTelemetry · prometheus-community 등의 Helm 차트에 Gateway API HTTPRoute 지원 추가, Go 프로젝트에는 버그 수정 기여

인프라 보안 & IAM

Keycloak Operator 기반 중앙 인증(SSO) 도입과 Vaultwarden 비밀번호 관리 시스템으로 사내 인증·시크릿 거버넌스 일원화

- Keycloak Operator + KeycloakRealmImport로 중앙 인증 시스템 도입 — 기존 GitLab 계정 그대로 로그인, ArgoCD / Harbor / Vaultwarden OIDC 통합(검증 PASS), GitLab group → Keycloak mapper → ArgoCD role 단일 권한 흐름 확립
- Vaultwarden을 Kubernetes에 Helm 배포, GitLab SSO + 자동 백업(SQLite 덤프 스케줄링) + 대화형 restore 스크립트로 사내 시크릿 중앙 관리

주요 경력 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, AWS 기반 클라우드와 온프레미스 서버를 설계·구축·운영하고 있습니다. dev·qa 서버는 온프레미스에, review·prod 서버는 AWS에 구축하여 하이브리드 아키텍처를 운영 중이며, 이 구조로 월 인프라 비용을 20%(\$60,000 → \$48,000) 절감했고, 그 절감분은 신규 프로젝트 인프라에 재투자됐습니다.

현재 서비스 중인 라이브 게임에 대해 외부 퍼블리셔와 협업하여 2차 기술지원 및 운영을 담당하고 있으며, 개발 중인 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도: 하이브리드 아키텍처 설계·구축 100% (단독), 라이브 게임 2차 기술지원·운영은 외부 퍼블리셔 운영팀과 협업 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영하던 중 월 \$60,000의 높은 클라우드 비용이 발생하여 비용 최적화가 필요했습니다.

접근 전략

워크로드별 비용을 분석한 결과, dev·qa 환경은 트래픽 변동이 적어 클라우드의 자동 스케일링이 불필요했고, review·prod 환경만 확장성과 안정성이 요구되는 상황이었습니다. 이에 dev·qa 환경은 온프레미스로 전환하고, review·prod 환경은 AWS에 유지하는 하이브리드 아키텍처를 설계했습니다. 양쪽 환경은 VPN 없이 역할을 분리해 독립 운영(앱 트래픽 분리)하며, 이미지 레지스트리도 환경별로 분리(dev·qa=Harbor, review·prod=ECR)해 아키텍처 차이에 맞춰 각 환경에서 독립적으로 빌드했습니다. 양쪽 환경 모두 Terraform과 Ansible을 활용한 IaC 통합으로 운영 표준을 일원화(신규 컴포넌트 IaC 100%)하여, 운영 복잡도 증가 없이 일관된 인프라 관리 체계를 확립했습니다. IAM은 서비스·CI 계정만 코드로 관리하고 사람 계정은 원칙적으로 코드 밖에 두되, 콘솔 접근이 필요한 개발자 계정은 Terraform이 액세스 키를 만들지 않고 각자 발급·회전하도록 설계해 장기 자격증명이 state에 남지 않게 했습니다.

트러블슈팅

- EKS 환경에서 ALB Ingress 구성 시, 게임 클라이언트 버전별로 다른 백엔드로 라우팅해야 하는 요구사항 발생. ALB의 조건부 라우팅 규칙에서 커스텀 헤더 기반 라우팅과 Target Group을 조합하여 해결
 - ArgoCD로 관리하는 프로덕션 서비스의 rolling deploy 중 ALB target deregistration delay(대상 등록 해제 대기)와 Pod SIGTERM 처리 시점이 어긋나 일부 세션 connection drop 발생. 3가지 조치로 connection drop 0건 달성 — (1) ALB drain 타이밍 정렬: preStop hook + readiness 우선 fail로 종료 중 신규 요청은 차단하되 in-flight 요청은 끝까지 처리, (2) RollingUpdate 전략을 replica 스케일에 맞춰 surge-first로 조정해 롤아웃 중 용량 유지, (3) 다중 replica 서비스에 PDB를 적용해 Karpenter consolidation(노드 통합·축소) 시 Pod 동시 축출 방지.
- 확인: CloudWatch 기반 Grafana 대시보드로 ELB 5xx와 Target 5xx를 분리해 보고, target group의 healthy/unhealthy host 수와 p95 응답시간을 배포 전후로 대조

성과

- 워크로드 분석 기반 인프라 최적화로 월 비용 20% 절감 (\$60,000 → \$48,000), 연간 약 \$144,000 — 절감분은 신규 프로젝트 인프라에 재투자됨
- 환경별로 역할을 분리해 장애를 격리하고 운영 안정성 확보

프로젝트 2: CI/CD 파이프라인 구축 및 고도화

기여도 100% (인프라 및 파이프라인 전체 담당) 2024.12 ~ 현재 (운영 중)

문제

개발자가 수동으로 빌드 후 서버에 배포하는 방식으로, 배포 시 평균 30분이 소요되었으며 휴먼 에러로 인한 장애가 빈번하게 발생했습니다.

접근 전략

GitLab CI와 ArgoCD를 조합한 GitOps 기반 자동 배포 환경을 구축했습니다. 개발자가 Git Push만 하면 빌드→테스트→배포가 자동으로 진행되고, ArgoCD가 클러스터 상태를 계속 지켜보다가 Git에 선언된 상태와 달라지면 그에 맞춥니다. 이미지 빌드는 Kaniko로 처리했습니다. CI에서 Docker 데몬으로 빌드하려면 러너에 권한 있는(privileged) 컨테이너를 띄워야 하는데(Docker-in-Docker), Kaniko는 그 없이 빌드하므로 이 방식을 택했습니다. 멀티 환경(dev/qa/review/prod) 배포는 단일 파이프라인에서 관리하도록 구성했습니다.

트러블슈팅

- 잘못된 커밋 하나로 하위 Application이 통째로 지워질 수 있는 문제 → 자동 삭제(`prune`)를 계층별로 분리. 워크로드 Application은 `prune: true` + `selfHeal: true` 로 드리프트를 즉시 복구하고, app-of-apps 루트와 AppProject 계층만 `prune: false` 로 두어 삭제는 사람이 확인.
- 모든 sync 이벤트는 Slack 알림으로 추적성 확보
- 데이터 업로드 경로에서 SSH 키를 제거 — 기존에는 배포 쪽이 SSH로 대상 인스턴스에 직접 접속해 파일을 복사했고(scp/rsync), 그래서 배포 계정이 SSH 키를 들고 있어야 했습니다.
- 이 경로를 없애고 파일을 S3 중계 버킷에 올린 뒤, SSM SendCommand(SSH 없이 원격 명령 실행)로 대상 인스턴스가 스스로 S3에서 내려받도록 바꿔 배포 계정에서 SSH 키를 완전히 없애고 키 유출 리스크를 차단

성과

- 배포 시간 67% 단축 (30분 → 10분)

- 데이터 배포에서 소스를 받아오는 단계를 약 139초 → 약 30초로 단축 — 기존에는 `git pull` 로 받아오던 것을, 최신 커밋만 알게 받고 파일은 실제로 필요할 때 가져오는 방식(shallow + blobless fetch)으로 교체
- GitOps 자동 배포로 머지 기반 지속 배포 체계 확립 — 핵심 서비스 repo당 주 100회 내외로 배포되며, 신규 피처가 사용자에게 달기까지 걸리는 시간을 단축
- 빌드·배포·설정 적용의 수동 단계를 파이프라인으로 대체해 휴먼 에러로 인한 배포 장애를 제거하고, cross-repo MasterData 파이프라인으로 데이터 변경 시 하위 서비스로 자동 전파(fan-out) + 순환 트리거(loop) 원천 차단

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석까지 전 과정 단독 설계·구축) 2024.11 ~ 현재

로그가 분산된 온프레미스 환경에서 시작해 중앙 집중식 로깅을 3단계로 고도화(ELK → EFK → ECK Operator)하고, 수집기 데이터 무결성·HA를 강화한 뒤, 축적된 EFK 로그 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 확장했습니다. build → operate → improve → analytics로 이어지는 관측 플랫폼 전 과정을 단독으로 설계·운영했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

문제

온프레미스 환경에서 서버 및 컨테이너 로그가 분산되어 있어 장애 발생 시 원인 파악에 많은 시간이 소요되었으며, 통합 모니터링 체계도 없었습니다.

접근 전략

로깅 스택을 **ELK → EFK → ECK Operator 3단계로 고도화**했습니다. 초기 ELK는 노드마다 Filebeat가 로그를 수집하고 중앙 Logstash가 가공하는 구조였는데, JVM 기반 Logstash 가공 티어의 메모리 부담이 커 수집은 경량 Fluent Bit DaemonSet으로, 가공은 Fluentd로 교체한 EFK로 전환했습니다. 이후 Elastic 공식 Helm 차트가 2년 이상 정체되자 CRD 기반 선언형 관리(ECK Operator)로 옮기며 Elastic Stack을 8.5.1 → 9.0.0으로 major 업그레이드했고, 온프레미스 EFK 스택을 AWS(EKS)용 `-aws` 스택으로 템플릿화해 같은 구조를 이식하면서 로그 수명주기(ILM)와 S3 스냅샷 보관을 IRSA로 정적 키 없이 자동화했습니다.

트러블슈팅

- 온프레미스 스택에서 Elastic Stack 신버전 자동 추종 중, 아티팩트 피드에는 올라왔으나 이미지가 아직 퍼블리시되지 않은 버전을 당겨 `ImagePullBackOff` → 되돌리려 하자 ECK admission webhook이 버전 하향을 거부(`Downgrades are not supported`)해 롤백 경로가 막히고 operator까지 죽음.
깨지는 것보다 **되돌릴 수 없는 것**이 실제 위험이라 보고, 업그레이드 스크립트에 이미지 퍼블리시 확인·헬스 사전 점검·major 상향 시 스냅샷 경고·webhook 제약을 감안한 복구 절차를 추가

성과

- 로그 검색을 개별 Pod 접속 확인에서 Kibana 단일 인터페이스로 통합, MTTR 개선으로 서비스를 안정적으로 유지
- 무거운 Logstash를 경량 Fluent Bit + Fluentd 구조로 재설계하고, ECK Operator로 전환해 TLS 인증서·계정·업그레이드를 수동으로 관리하던 부담을 제거(CR spec.version 한 줄 변경으로 롤링 업그레이드), 2년간 정체된 Elastic Stack의 major 업그레이드를 완료 (8.5.1 → 9.0.0)

3-2. Fluent Bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 2026.05)

Fluent Bit Pod 재시작 시 tail offset 소실로 인한 로그 재색인·누락 위험을 SQLite offset DB로 해결 — DaemonSet은 노드마다 Pod가 뜨므로 offset을 노드별 로컬 경로에 두어 저장소 수명을 노드 수명과 일치시킴, 재색인 0건·중복 0건, 수집기가 조용히 멈춘 상태를 감지하는 alert로 미감지 장애 차단

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

장애·검색용 EFK 스택을 Elasticsearch Transform(5분)으로 사용자당 1행 집계하는 제품 분석 플랫폼으로 확장 — Kibana 대시보드로 D+1~D+30 리텐션·코호트 제공, 별도 SaaS 없이 구현

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% (전체 시스템 설계 및 개발) 2025.02 ~ 현재

개발팀의 반복적인 수동 작업을 자동화하고 생산성을 높이기 위해, 5가지 사내 도구를 설계·개발·운영했습니다.

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

Jenkins APK 빌드 완료 시 Slack으로 완료 메시지·다운로드 QR을 자동 전송하는 Python 봇을 Kubernetes에 배포 — QA 전달 시간 90% 단축, 버전 혼동 제거

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab push를 트리거로 문서를 Google Drive에 rclone으로 단방향 동기화 — `--checksum` 비교로 실제 변경분만 전송, 문서 관리 시간 80% 단축(10분 → 2분), GitLab이 단일 기준이고 Drive 쪽 수정은 다음 동기화에서 덮어씀

4-3. 사내 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama·Open WebUI를 docker compose로 올려 사내 LLM 직접 운영 — 코딩 모델을 상주시켜 개발팀에 열어 둠

4-4. Git 저장소 미러링 도구 개발 + Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 2026.06)

CodeCommit·GitLab·GitHub 수동 동기화의 누락·지연을 없애려 Go로 양방향 Git 미러링 도구 git-bridge를 만들어 오픈소스 공개 — 저장소 10개·일 평균 30건 100% 자동화, webhook secret 노출 없는 Retry API 복구 수단과 `ref_overrides` 방향 고정으로 이력 되돌림 차단(테스트 전용 repo 6/6 PASS)

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

[halverneus/static-file-server](#)가 디렉토리 UI·검색·ZIP 일괄 다운로드 없이 파일만 내려주고 업스트림 유지보수도 멈춰, Go로 정적 파일 서버를 직접 만들어 Apache-2.0으로 공개하고 자체 Helm chart·NFS로 배포해 완전 대체 — 일 평균 30건 다운로드·주 5회 APK 배포 처리

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% (단독 설계 — 모니터링·업그레이드 자동화·Helm 관리 프레임워크·NGF 마이그레이션·OSS Helm 차트·스토리지 성능 최적화·Shell→Python 마이그레이션, 자동화 스크립트로 표준 절차를 코드로 남겨 → 팀 재사용 가능) 2025.12 ~ 현재

클러스터를 안정적으로 운영하기 위해 모니터링을 개편하고 K8s 업그레이드·Helm 관리를 자동화했으며, 네트워크 컨트롤러를 Gateway API 기반으로 마이그레이션했습니다.

NGF 마이그레이션과 ECK Operator 도입의 산출물을 OSS Helm 차트로 공개하고, control-plane / DB 노드의 SSD 마이그레이션 및 빌드 I/O 격리로 스토리지 성능 병목을 해소했으며, 인프라 배포/업그레이드 파이프라인을 단계별(wave) 순차 자동화 + Shell → Python 마이그레이션으로 정비했습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack 기반 커스텀 알림·Grafana 대시보드를 설계하고 DB·node-exporter·Cilium 메트릭을 통합 — 커스텀 알림을 컴포넌트 그룹별로 묶어 운영하고, 심각도(severity)별로 나눠 보내며 상위 알림이 뜨면 하위 알림을 억제(inhibit)해 같은 장애의 중복 통보 제거

5-2. K8s 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespray 업그레이드 절차와 업그레이드 후 9개 항목 검증 스크립트(노드·etcd·API Server·인증서 만료 등)를 만들고 Helm 프레임워크·표준 템플릿 동기화로 버전 관리를 표준화 — 검증 시간 90% 단축, 인증서 갱신을 스크립트화해 만료 장애 제거

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

온프레미스 ingress-nginx 다수 인스턴스를 NGF 2.x 단일 컨트롤플레인+클래스별 Gateway CR로 전환 — MetalLB IP 유지로 DNS·방화벽은 그대로 두고 사고 없이 cutover(클래스당 30~60초), wildcard 인증서로 수동 갱신 50% 감소

5-4. OSS Helm 차트 공개 — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

NGF·ECK 산출물을 범용 차트(nginx-gateway-cr·elasticsearch-eck·kibana-eck)로 다듬어 Apache-2.0으로 ArtifactHub에 공개 — mgmt 클러스터를 cluster_uuid를 그대로 유지한 채 공용 OCI 차트로 전환

5-5. 스토리지 성능 최적화 — etcd / DB SSD 마이그레이션 + 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

GitLab Runner buildx의 디스크 I/O 급증으로 HDD 위의 etcd·게임 DB가 초 단위 지연을 겪던 것을 진단해 control-plane·DB 워커를 SSD로 이전하고, 별도 물리 서버의 빌드 전용 VM에 buildx를 격리 — 게임 DB COMMIT 9~14초 → ms 수준 회복, 빌드 중 플레이 불가 장애 0건

5-6. 인프라 배포/업그레이드 자동화 + Shell → Python 마이그레이션 (2026.03 ~ 현재)

문제

사내 Kubernetes 인프라 monorepo를 helmfile + shell로 직접 자동화해 운영해왔는데, 컴포넌트가 늘면서 shell + awk로는 환경별 분기와 복잡한 YAML 수정을 감당하기 어려워졌고 테스트도 불일 수 없었습니다. 신규 배포·차트 업그레이드·template 동기화도 여전히 수동이었습니다.

접근 전략

CI 파이프라인을 6단계 wave(bootstrap → core → security → db-redis → observability → apps)로 나누고, MR 변경분을 감지해 바뀐 컴포넌트만 helmfile로 배포하되 프로덕션 반영 전에는 사람이 승인하도록 했습니다. 차트 신버전이 나오면 CI가 이를 감지해 무엇이 바뀌었는지·설정값 차이·호환성이 깨지는 변경을 정리한 MR을 자동으로 열어주도록 업그레이드도 자동화했습니다.

기존 shell + awk 자동화는 외부 패키지 없이 표준 라이브러리만 쓰는 Python으로 옮기며 표준 템플릿·동기화·백업·CI 스크립트를 모듈로 나누고 컴포넌트별 업그레이드 모듈과 단위 테스트를 붙였습니다(모듈마다 테스트 파일 1:1, 외부 러너 없이 `unittest discover`로 실행). CI에는 프로젝트 7의 helmfile-tools 이미지를 적용했습니다.

성과

- 컴포넌트 단계별(wave) 순차 자동 배포 + 업그레이드 MR 자동 생성 구축으로 인프라 배포/업그레이드 운영 부담 대폭 절감
- Shell + awk 자동화를 Python으로 마이그레이션 — 환경별 분기·복잡한 YAML 수정을 shell로 감당하기 어렵던 한계를 견어내고 가독성·테스트 가능성을 확보.
외부 패키지 없이 표준 라이브러리만으로 구현해, python3만 있으면 별도 설치나 가상환경 없이 어디서든 그대로 실행
- 로컬(macOS venv)과 원격(CI container)이 같은 코드를 실행하도록 보장해 개발 → 배포 사이의 환경 차이 문제 제거

5-7. helmfile apply(push 방식) → ArgoCD Pull-GitOps 전환 (App-of-apps 컨트롤플레인, 2026.06 ~ 현재)

문제

플랫폼 릴리스를 CI 러너에서 `helmfile apply` 로 클러스터에 직접 반영하는 push 방식이다 보니 배포 주체가 CI 러너에 묶여 있었고, 멀티클러스터로 확장하면서 클러스터별 상태 드리프트를 선언적으로 바로잡을 pull 기반 GitOps 컨트롤플레인이 필요했습니다.

접근 전략

CI 러너의 `helmfile apply` push 배포를 ArgoCD pull-GitOps로 전환했습니다. Matrix + Git files 제너레이터 기반 ApplicationSet이 각 컴포넌트의 `argocd/*.yaml` 마커 파일을 읽어 Application을 자동 생성하도록 구성해, 온프레미스와 AWS의 플랫폼 릴리스를 양쪽 클러스터에 등록했습니다. 기존 helm 릴리스는 재생성 없이 그대로 ArgoCD 관리로 넘겨받아(adoption) 배포 순서를 보존했고, App-of-apps 계층에 연쇄 삭제 방지(prune-cascade) 안전장치와 AppProject 최소 권한을 적용해 실수로 인한 대량 삭제 리스크와 장애 영향 범위(blast-radius)를 축소했습니다.

성과

- `helmfile apply` push 배포 → ArgoCD pull-GitOps 전환으로 온프레미스·AWS 클러스터의 플랫폼 릴리스를 단일 컨트롤플레인에서 선언 관리, 기존 helm 릴리스를 재생성 없이 adopt해 배포 순서 보존
- App-of-apps 연쇄 삭제 방지(prune-cascade) 안전장치로 GitOps 대량 삭제 리스크 차단, AppProject 최소 권한 whitelist로 장애 영향 범위(blast-radius) 축소

프로젝트 6: 인프라 보안 체계 구축

기여도 100% (설계, 배포, SSO 연동) 2026.04 ~ 현재

인프라 운영에 필요한 시크릿 관리, 중앙 인증, 원격 접속을 순차적으로 구축하고 있습니다.

6-1. Vaultwarden 비밀번호 관리 시스템 (1주, 2026.04 ~ 현재)

Slack·이메일·개인 저장소에 흩어져 있던 사내 시크릿을 Vaultwarden으로 중앙화 — Kubernetes에 Helm으로 배포하고 GitLab SSO(OIDC)·RBAC 연동, SaaS 비용 0원

6-2. Keycloak Operator 기반 중앙 인증(SSO) 시스템 도입 (2026.04 ~ 현재)

문제

ArgoCD·Harbor·Vaultwarden 등 사내 도구가 각각 GitLab을 OIDC IdP로 직접 물어 신원 소스가 도구마다 하드코딩된 상태였습니다. GitLab은 OIDC를 부가 기능으로만 제공해 LDAP 같은 외부 신원 소스나 클라이언트별 인증 정책을 엮기 어려웠고, 소스를 바꾸려면 도구를 하나씩 다시 설정해야 했습니다. 로그인 브로커(Keycloak)를 앞에 세워 도구를 신원 소스에서 분리하기로 했습니다.

접근 전략

Keycloak Operator + KeycloakCR + KeycloakRealmImport를 PostgreSQL 백엔드와 함께 도입해 Keycloak이 GitLab 로그인을 중개하도록 구성하고, 사용자는 기존 GitLab 계정 그대로 로그인합니다. Realm / Client / IdP / Group / Mapper 설정을 부트스트랩 스크립트로 코드화하고, Harbor·ArgoCD·Vaultwarden의 OIDC를 GitLab 직접 연동에서 Keycloak 경유로 전환했습니다. RBAC는 GitLab group → Keycloak group mapper → ArgoCD role로 연결해 권한 기준을 Keycloak으로 일원화했습니다.

성과

- ArgoCD / Harbor / Vaultwarden OIDC 통합 완료, 검증 PASS — GitLab 직접 연동 의존성 해소
- GitLab group(`server` , `global-admin`) → Keycloak group mapper → ArgoCD role 권한 흐름을 Keycloak 단일 기준으로 일원화, 향후 LDAP 연동·MFA 강제·세션 정책 중앙화의 기반 마련
- 도입 중 부딪힌 시행착오(IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping)를 설계 문서에 정리해 클러스터 재구축 시 재발 방지

6-3. GitOps 시크릿 관리 — External Secrets Operator · Sealed Secrets (2026.05 ~ 현재)

DB 비밀번호를 Helm values 평문으로 Git에 커밋하던 구조를 견어내고, External Secrets Operator로 AWS Secrets Manager를 연결해 ExternalSecret 참조로 전환, Harbor pull secret은 cluster-wide SealedSecret으로 표준화 — AWS SM을 시크릿의 단일 관리 기준으로 일원화

6-4. OpenVPN 원격 접속 서버 (2026.07)

물리 서버에 OpenVPN 원격 접속 서버를 직접 구축해 동시 접속 최대 50 규모의 사내망 원격 접속 환경을 표준화 — easy-rsa로 사내 자체 인증서 체계(PKI)를 세워 접속자별 EC 인증서(prime256v1)를 발급하고, tls-crypt로 제어 채널까지 암호화해 정상 인증서가 없는 시도는 연결 협상 단계에서 차단(통신은

AES-256-GCM-TLS 1.2+).

기본 full-tunnel에 split-tunnel 전환을 열어 두고, 설치와 인증서 발급·폐기를 먹등 스크립트로 자동화

프로젝트 7: 공용 Toolchain Image 계층 구조 + 자동 버전 사이클

기여도 100% (아키텍처 설계 · CI 표준화 · 자동화 파이프라인 개발) 2025.09 ~ 2026.05

문제

GitLab CI 표준화 범위 안의 repo들에서 각 잡이 alpine 베이스에 helm·helmfile·kubectl·crane·aws-cli 등을 매번 설치하면서 첫 번째 잡 준비 시간이 평균 5분에 달했고, 도구 버전이 consumer repo마다 흩어져 있어 신버전 추가 시 모든 consumer repo를 함께 손대야 했습니다. `:latest` 태그 의존으로 재현성이 떨어지고 취약점 추적도 어려웠습니다.

접근 전략

Layer 1 (Mirror): 외부 레지스트리(Docker Hub, GHCR)의 도구 이미지를 `crane copy` 로 사내 Harbor에 multi-arch manifest(여러 CPU 아키텍처를 묶은 이미지 목록)를 보존한 채 미러링.

Layer 2 (Custom): Layer 1 위에 도구를 미리 설치해 둔 대용량 이미지(helmfile-tools, base-tools, node-build, rclone-backup, aws-cli-tools)를 빌드해 consumer repo가 image pin만으로 도구를 즉시 사용할 수 있도록 표준화.

Tier 1~2 SSOT(단일 관리 기준): `.toolchain_build_custom` 공용 template + `TOOLCHAIN_*_TAG` 중앙 변수를 도입해 consumer repo가 image tag를 직접 추적하지 않고 중앙 변수만 바라보도록 분리.

공용 CI 템플릿 SSOT: repo마다 복사해 쓰던 CI 잡을 중앙 공용 CI 템플릿 repo로 추출해 각 repo가 `include` 로 참조하도록 전환. auth(정적 키 없는 AWS 인증) / build(kaniko) / deploy(ArgoCD) / backup(Google Drive) 잡 템플릿과 공용 앵커(git·ssh·rules·job-config·packages)를 단일 소스로 관리해 복사본마다 설정이 갈라지는 문제를 제거.

자동 버전 사이클: 주간 스케줄이 (1)을 돌려 신버전을 감지하고, 사람이 파이프라인을 실행하면 (2)~(4)가 자동 MR로 이어집니다. 사람이 판단하는 지점은 파이프라인 실행과 Tier 2 MR 머지 두 곳입니다:

- (1) upstream 신버전 감지
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 TOOLCHAIN_*_TAG 동기화 auto-MR
- (4) consumer repo(template provider repo 제외)에 자동 전파

트러블슈팅

- 도구 신버전을 자동으로 따라 올리다가 helmfile 1.5.10이 helm 3.18.6 이상을 요구하는데 helm은 3.16.4에 핀돼 있어 `build_custom` 빌드가 깨짐 → 도구 간 호환 요구사항은 upstream 릴리스 노트를 봐야만 알 수 있어 자동 감지가 불가능하다고 판단, helm·helmfile·kubectl에 minor-guard를 도입해 현재 minor 안의 patch만 자동 상향하고 minor·major 상향은 사람이 호환성을 확인한 뒤 올리도록 분리(`*_TARGET_MINOR` 환경변수로 계획된 minor bump 시 1회 override)

성과

- CI 첫 번째 잡 준비 시간 5분 → 약 10초 (약 95% 단축), 사내 repo 전반에 도구 버전 일관성 100% 확보, 버전 거버넌스 4-tier(ARG → 중앙 변수 → consumer 분리 → lint drift 감지)로 흩어져 있던 버전 지정 지점을 한 흐름으로 정리
- helm·helmfile·kubectl·crane·rclone 신버전 전파를 자동화하되, 파급이 큰 이미지 리빌드는 스케줄에 맡기지 않고 사람 검토를 거치도록 게이트를 뒤 판단 지점을 두 곳으로 좁힘
- 컴포넌트를 리스크 티어(T1 주간 / T2 격주 / T3 월간 + pre-flight)로 분류한 자동 업그레이드 파이프라인 구축(신버전 감지 → 자동 MR → Slack 알림 → wave 순차 적용, MAJOR_PIN 가드·T3 atomic 롤백), docker buildx로 amd64 + arm64 multi-arch 이미지를 빌드하고 외부 이미지는 crane으로 Harbor에 미러링해 외부 레지스트리 의존 제거

프로젝트 8: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (그린필드 EKS 단독 설계·구축) 2026.06 ~ 현재 (구축·운영 중)

문제

라이브 게임 환경(AWS review·prod)과 별개로, 신규 외부 퍼블리싱 게임은 온프레미스 개발 클러스터 하나로만 운영 중이었고, 프로덕션 런칭을 위해 확장성·가용성을 갖춘 별도 AWS EKS 프로덕션 환경이 필요했습니다. 온프레미스만으로는 글로벌 트래픽 대응과 오토스케일링에 한계가 있었습니다.

접근 전략

eu-central-1 리전에 그린필드 EKS 클러스터를 구축해 온프레미스 개발 + AWS 프로덕션의 멀티클러스터 하이브리드로 확장하고, 플랫폼 컴포넌트를 전부 GitOps(ArgoCD)로 선언 관리했습니다.

오토스케일링·네트워킹: Cluster Autoscaler 대신 Karpenter를 채택해 워크로드별로 NodePool을 둘로 나눴습니다 — CI 빌드는 ARM 다중 family 스팟 (독립 capacity pool 수를 늘려 빌드 회수 빈도를 낮춤), 게임은 on-demand로 구성하고, AWS Load Balancer Controller(ALB/NLB) + ExternalDNS(Route53) + wildcard ACM으로 인그레스·DNS·TLS를 자동화했습니다.

로깅·시크릿: eck-operator 기반 EFK 스택(Elasticsearch 3노드 3-AZ, EBS gp3, S3 스냅샷 SLM)을 HA로 구성하고, External Secrets Operator로 시크릿을 외부화했습니다. 모든 ServiceAccount 인증은 IRSA / Pod Identity로 처리해 정적 키를 제거했습니다.

인증·배포: ArgoCD에 GitLab OAuth SSO + 게임 스코프 RBAC를 연동하고, 클라이언트 아티팩트는 Jenkins → S3 + CloudFront로, 서버 매니페스트는 S3 transit 버킷 + SSM SendCommand(SSH 없이 원격 명령 실행)로 bastion의 EFS 마운트에 전달했습니다. bastion을 Name 태그로 동적 탐색해 SSH 키 없이 배포했습니다.

트러블슈팅

- 라이브 세션·Apple 심사 중 노드 회수가 없도록 게임 워크로드는 on-demand NodePool로 분리 → consolidation은 막지 않고 허용. stateless HTTP(세션 Redis 외부화)라 preStop drain + PDB `minAvailable: 1` + budget `nodes: 1` 로 한 번에 한 노드만 비워 Ready 1 미만 유지. `do-not-disrupt` 는 축소를 막아 미사용
- 롤아웃을 재시작할 때마다 노드가 늘어난 뒤 줄지 않는 문제 → Karpenater consolidation 정책과 롤링 배포 `maxSurge` 가 맞물려 새로 뜬 노드가 회수되지 않고 남던 것이 원인.
노드별 메모리 request 점유율을 실측해(한 노드 92% / 다른 노드 35%) 특정 워크로드의 request가 과다 산정된 것을 찾아 2Gi → 512Mi(실측 사용량 약 101Mi)로 조정하고 consolidation 조건을 함께 바꿔 bin-packing 회복
- 롤링 배포 중 ALB가 아직 준비되지 않은 Pod로 트래픽을 전달하는 문제 → ALB Pod readiness gate + PodDisruptionBudget + graceful drain(연결을 안전하게 종료)을 조합해 해결

성과

- 온프레미스 개발 클러스터 하나 → 온프레미스 + AWS 프로덕션 멀티클러스터 하이브리드로 확장, 신규 외부 퍼블리싱 게임의 클라우드 프로덕션 런칭 기반 확보
- 플랫폼 컴포넌트를 100% GitOps(ArgoCD)로 선언 관리하고, IRSA / Pod Identity + SSM SendCommand로 정적 키 한 장 없이 인증·배포하도록 정리해 bastion SSH 키 유출 리스크 제거
- 스팟 NodePool을 ARM 다중 인스턴스 family로 넓혀 독립적인 스팟 capacity pool 수를 늘림 — 합산 중단 확률을 낮춰 진행 중인 빌드가 회수되는 빈도를 줄이는 것이 목적이고, 인스턴스 크기는 고정해 노드 단가 구조는 유지.
프로덕션과 리뷰 워크로드는 NodePool을 나누지 않고 하나로 묶어 bin-packing — 풀을 나누면 풀마다 최소 1노드를 점유하는 바닥이 생기기 때문
- ArgoCD 앱 마커와 프로덕션 CI/CD 전환 문서로, 단독 구축이지만 팀이 동일 절차로 운영·재구축할 수 있도록 인수인계까지 문서로 남김

프로젝트 9: Observability 고도화 — 분산 추적 & 무중단 배포 전략

기여도 100% (단독 설계·구축 — OTEL 추적 파이프라인·Argo Rollouts Canary·소프트 런칭 운영) 2026.06 ~ 현재

Project 8의 AWS EKS 프로덕션 게임 런칭 플랫폼 위에, 신규 외부 퍼블리싱 게임의 소프트 런칭에 대비해 관측과 배포를 보강했습니다.

OpenTelemetry 기반 엔드투엔드 분산 추적 파이프라인으로 로깅·메트릭만 있던 기존 관측 플랫폼에 '추적' 축을 채우고, Argo Rollouts로 Canary 배포를 도입했으며, 이 스택 위에서 소프트 런칭 실패트래픽을 안정적으로 받아냈습니다.

9-1. OpenTelemetry 엔드투엔드 분산 추적 파이프라인 (OTel + Tempo, 2026.06 ~ 현재)

문제

기존 중앙 관측 플랫폼은 로깅(EFK)과 메트릭(kube-prometheus-stack)만 갖추고 '추적(trace)' 축이 비어 있어, 요청이 서비스 사이를 어떻게 흐르는지, 어디서 지연이 발생하는지 확인할 수 없었습니다. 프로덕션 런칭을 앞두고 서비스 간 지연·에러의 원인을 분석할 수단이 필요했습니다.

접근 전략

AWS EKS에 OpenTelemetry Collector + Grafana Tempo(S3 저장, EKS Pod Identity로 정적 키 없음) + OpenTelemetry Operator(auto-instrumentation으로 앱 이미지 변경 없이 언어 SDK 주입)로 분산 추적 스택을 구축했습니다. 여기서 핵심은 샘플링 지점을 나눈 것입니다. RED 메트릭은 샘플링 이전 100% span에서 생성해 정확도를 확보하고, trace 저장 단계에만 tail sampling(모든 에러 + 500ms 초과 + 나머지 10% 보존)을 적용해 추적 저장 비용을 절감했습니다. 마지막으로 Grafana에서 Metrics ↔ Traces ↔ Logs 3-signal을 서로 오갈 수 있도록 연결했습니다(기존 로깅·메트릭 플랫폼은 Project 3).

성과

- Metrics ↔ Traces ↔ Logs 3-signal을 Grafana에서 서로 오갈 수 있게 연결해, 요청이 서비스 사이를 어떻게 흐르고 어디서 지연되는지 추적 가능해짐
- RED 메트릭 생성(샘플링 이전 100% span)과 trace 저장(tail sampling)을 분리해 정확도는 지키고 저장 비용은 절감
- auto-instrumentation으로 앱 이미지 변경 없이 언어 SDK를 주입하고, EKS Pod Identity로 정적 키 없이 S3에 접근

9-2. Argo Rollouts 기반 무중단 배포 전환 (Blue-Green → Canary, 2026.06 ~ 현재)

문제

프로덕션 게임 서비스는 배포 중에도 라이브 트래픽을 끊으면 안 되고, 문제가 생기면 즉시 되돌릴 수단이 필요했습니다.

그런데 당시 쓰던 Blue-Green은 승격할 때 ALB target group을 한 번에 통째로 교체하는 방식이라, 그 순간 healthy target이 0이 되고, 그 짧은 공백 동안 ALB가 503을 반환했습니다. old/new가 안전하게 공존하며 넘어가도록 전환 과정을 통제할 방법이 필요했습니다.

접근 전략

Argo Rollouts Canary로 전환했습니다. 같은 ALB target group에서 `maxUnavailable: 0` + add-before-remove(새 Pod를 먼저 등록한 뒤 구 Pod를 제거)로 old/new가 순간 공존하며 넘어가 503 레이스를 제거했고, 게임 서비스가 stateless HTTP(JWT + Redis/TypeORM로 상태 외부화)라 전환 중

old/new에 트래픽이 섞여 들어와도 안전합니다.

공용 base-aws Helm 차트에서 배포 전략을 서비스별로 선택해서(opt-in) 켜고 끄도록 템플릿화해, 프로덕션 게임 메인 서비스 1개만 Canary로 전환하고 (공개 트래픽이 없는 형제 내부 서비스는 일반 Deployment 유지), 컨트롤러는 leader-election HA + PDB로 구성해 노드 drain·Karpenter consolidation 중에도 롤아웃이 멈추지 않습니다. Pod가 들어올 때와 나갈 때는 트래픽이 끊기는 이유가 달라 장치를 돌로 나눴습니다.

들어올 때는 네임스페이스 라벨로 ALB Pod readiness gate를 주입해 ALB target group에 정상 등록되기 전까지 Pod를 Ready로 세지 않게 했고, 그래야 `maxUnavailable: 0` 이 지키는 수가 실제로 트래픽을 받을 수 있는 Pod 수와 같아집니다. 나갈 때는 ALB deregistration delay 30초 → preStop 35초 → terminationGracePeriodSeconds 60초로 짧은 것부터 정렬했고, SIGTERM에서 스스로 in-flight를 정리하는 .NET 서비스만 75초입니다.

다만 old/new가 동시에 떠 있으면 안 되는 변경(API 스펙 변경 등)은 이 전략으로 덮을 수 없어, 게임 서비스 특성상 정기점검 시간에 처리합니다.

성과

- 프로덕션 게임 서비스를 Blue-Green에서 Canary로 전환해, 승격 시 ALB의 healthy target이 순간 0이 되며 발생하던 503 레이스 제거
- 하위 호환되는 패치는 old/new가 순간 공존하며 넘어가 무중단으로 배포

9-3. 소프트 런칭 운영 (2026.07 ~ 현재)

성과

- 2026.07.14(KST) 소프트 런칭 개시 — 앞의 스택 위에서 신규 외부 퍼블리싱 게임을 제한 공개로 열어 그 소규모 실트래픽을 Canary 배포와 3-signal 관측으로 안정적으로 받아냄
- HPA·Karpenter 오토스케일링은 2026 Q4 글로벌 정식 런칭을 대비해 프로비저닝 완료(소프트 런칭 규모에서는 아직 스케일아웃 미발생)
- 소프트 런칭 실트래픽 14일치 실측을 근거로 워크로드 requests·limits를 재산정 — 파드당 관측된 피크를 기준으로 request를 정해 HPA 스케일아웃이 실제로 걸리도록 정렬. request를 과다 설정하면 사용률이 HPA 목표치 밑에 영구 고정돼 스케일아웃이 한 번도 트리거되지 않기 때문이며, 실제로 기존 값에서는 한 번도 걸리지 않았음
- 2026 Q4 글로벌 정식 런칭에 필요한 관측·배포·오토스케일링을 모두 갖춤

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임·블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며 AWS에서 GCP로 대규모 클라우드 마이그레이션을 총괄했습니다.

비용 최적화와 글로벌 네트워크 성능 확보를 목표로, 게임 점검 시간을 활용해 최소 다운타임으로 전체 서비스를 이전했습니다. 소스 저장소는 온프레미스를 GitLab, 프로덕션을 GitHub로 이원화해 관리했습니다.

또한 MongoDB·CloudSQL·GA·Dune 등 멀티 소스 데이터를 BigQuery로 모아 Google Sheets까지 자동 갱신하는 데이터 수집 파이프라인을 구축해, 분석가의 일 2~3시간 수동 작업을 없앴습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70% (나머지 30% 는 서버 개발팀의 application 측 마이그레이션 협업), CI/CD 파이프라인 구축 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용이 지속적으로 상승하고 있었고(월 \$10,000+), 특히 상시 가동되는 게임 워크로드를 Fargate로 돌리며 발생하는 파드 단위 과금이 큰 비중을 차지했습니다. 비용 절감과 멀티 리전 게임 서비스를 위한 글로벌 로드밸런싱 확보를 목표로 GCP 마이그레이션을 추진했습니다.

접근 전략

3단계 마이그레이션 전략을 수립해 단계별로 실행했습니다 — 1단계 개발 환경을 GCP에 먼저 구축·검증, 2단계 QA 환경 이전, 3단계 프로덕션 이전. 네트워크는 Shared VPC(Host Project / Service Project 분리)로 설계하고, GitHub Actions의 GCP 인증은 Workload Identity Federation으로 전환해, 서비스 계정 키 발급·순환을 OIDC 단면 토큰으로 대체하고 키 관리 부담을 없앴습니다. DNS는 서브도메인 위임(Hosting.kr → AWS Route53 → GCP Cloud DNS)으로 사전 검증한 뒤, 최종 NS 변경 시점에만 점검을 공지하고 cutover 했습니다.

컴퓨트는 AWS Fargate로 돌던 워크로드를 GKE 표준 노드 풀로 재배치했습니다. 상시 떠 있는 워크로드에서는 파드 단위 과금이 VM 단위 과금보다 불리해서, 같은 과금 구조인 GKE Autopilot은 후보에서 뺐습니다.

비용·보안 측면에서는 Filestore의 SSD 최소 2.5TB 문제를 Compute Engine + pd-balanced 1TB 디스크 기반 NFS 서버 자체 구축으로 우회하고, Cloud Armor로 리전 차단 + IP 화이트리스트 WAF(웹 방화벽) 정책을 구성했습니다.

Terraform으로 GCP 인프라를 IaC화(IAM 제외 100%)하고, 마이그레이션 완료 후 GitLab CI와 ArgoCD를 조합한 GitOps CI/CD 파이프라인을 구축해 Helm Chart로 환경별 설정을 표준화하고, 모든 배포를 Git 커밋으로 되짚을 수 있게 했습니다.

트러블슈팅

- AWS ALB 기반 라우팅을 GCP 글로벌 로드밸런서로 전환하는 과정에서, 백엔드가 모두 unhealthy일 때 AWS ALB는 fail-open으로 트래픽을 통과시키지만 GCP LB는 이를 차단하는 차이 때문에 트래픽 라우팅 문제 발생.
GCP NEG(Network Endpoint Group) 구조를 분석하여 GKE 워크로드에 맞는 헬스체크·백엔드 설정으로 재구성
- GKE Ingress + BackendConfig의 healthcheck requestPath 응답이 누락되어 503 에러 발생. 해당 경로 응답을 보장하고 ManagedCertificate / FrontendConfig 설정과 서로 맞춰 외부 진입 트래픽 정상화
- Cloud CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드가 실패.
URL Map을 HTTP / HTTPS 두 개로 분리(`default_url_redirect.https_redirect=true`) 하고 HTTP forwarding rule에 80 포트 target

HTTP proxy를 별도 매핑하여 해결

- DNS를 GCP Cloud DNS로 위임한 뒤 Google Search Console에 도메인이 자동 등록되지 않아 검색 노출·도메인 소유 확인에 문제 발생. Search Console이 발급한 verification TXT 레코드를 Cloud DNS에 추가해 도메인 소유권을 검증한 뒤 재등록하여 해결

성과

- AWS → GCP 마이그레이션을 설계·수행해 클라우드 비용 50% 절감 (월 \$10,000 → \$5,000), 연간 약 \$60,000 비용 절약 — Fargate로 돌던 워크로드를 GKE 표준 노드 풀로 재배치해 파드 단위 과금을 VM 단위 과금으로 바꾸고(같은 이유로 Autopilot은 배제), MSP 리셀러 할인을 확보
- 3단계 전략으로 전체 서비스 마이그레이션 완료. 리소스 복사 방식으로 다운타임 최소화하고, 최종 DNS 전환 시에는 게임 서비스 특성을 반영해 2시간 내외 점검 공지 후 전환
- 전체 GCP 인프라를 Terraform으로 IaC화하고, 배포 이력을 Git 커밋으로 100% 추적 가능하게 함
- Workload Identity Federation 도입으로 GitHub Actions의 GCP 서비스 계정 키 관리 부담을 제거하고 키 유출 리스크를 원천 차단, 단명 토큰 기반 보안 모델로 전환

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하다 보니 분석가가 매일 2~3시간을 소비했고, 휴먼 에러로 인한 데이터 누락도 빈번했습니다.

접근 전략

MongoDB · CloudSQL · Google Analytics · Dune 등 멀티 소스를 BigQuery로 적재하고 분석가용 Google Sheets까지 자동 갱신하는 데이터 파이프라인을 GCP 서비스 조합으로 구축했습니다. 소스별로 MongoDB는 Dataflow Flex Template ETL, CloudSQL은 BigQuery 직접 connection, GA · Dune은 각자 API(Dune은 별도 API Key)로 호출해 일관된 인덱스 스키마로 정규화했습니다. Python Cloud Function이 BigQuery 쿼리 결과를 Google Sheets API로 적재하고, Cloud Scheduler가 Daily(전일)·Monthly(전월) 두 주기로 자동 트리거하며, 별도 중복 제거 Cloud Function이 정기 실행돼 데이터 정합성을 유지합니다. 전체 인프라(BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM · Service Account)는 Terraform으로 IaC 관리해 환경 재구축과 변경 추적을 표준화했습니다.

트러블슈팅

- CloudSQL → BigQuery 직접 connection(federated query)이 리전 불일치와 connection Service Account 권한 부족으로 실패. connection을 동일 리전에 생성하고 connection SA에 Cloud SQL Client 권한을 부여하여 해결
- Dune API의 execute → poll → fetch 비동기 실행 구조와 rate limit으로 결과 누락·타임아웃 발생. 실행 상태 폴링 + 백오프로 결과 수신을 보장하고 API Key를 Secret Manager로 분리
- Daily 재실행 시 동일 행이 BigQuery에 재적재되는 비역등 문제 발생. 중복 제거 Cloud Function에 MERGE / ROW_NUMBER() 정합성 로직을 적용해 적재를 멍등화(재실행해도 결과 동일)하고 누락·중복 0 유지
- BigQuery 결과를 Google Sheets API로 적재할 때 단일 시트 1천만 셀 한도·write 쿼터와 분당 요청 한도(429)에 걸려 갱신이 실패. values.batchUpdate 체크 분할과 시트 분할로 셀 한도를 우회하고, 호출 pacing(throttle)·배치 처리·exponential backoff로 쿼터 내에서 동작하도록 구현

성과

- 데이터 수집 자동화 95% 달성 (일 2~3시간 수동 작업 제거)
- 자동화로 휴먼 에러 제거, 데이터 누락률 0% 달성
- 전체 인프라 Terraform IaC 화로 동일 파이프라인을 다른 GCP 프로젝트에 30분 내 재구축 가능, 변경 이력 100% Git 추적

프로젝트 3: AI 워크로드용 GPU 인프라 표준화 및 확장 (Windows → Linux · 세팅 자동화 · 사내 / GCP / 외부 IDC)

기여도 100% (인프라 설계·구축·운영) 6개월 (2024.02 ~ 2024.07)

문제

사내 GPU 서버가 Windows로 운영되고 있어서, 이미지 생성 모델(Stable Diffusion 계열)을 돌릴 때마다 엔지니어들이 실행 환경을 직접 세팅해야 했습니다. 드라이버·CUDA·프레임워크 조합을 손으로 맞추다 보니 같은 환경을 다시 만들기 어려웠고, 장비는 있는데 실제로 쓰기까지 시간이 걸렸습니다.

접근 전략

사내 GPU 서버를 Windows에서 Linux(Ubuntu 22.04)로 전환하고, 손으로 하던 세팅을 스크립트로 자동화했습니다. 드라이버 스택(nvidia-driver-535 · CUDA 11.5 · cuDNN 8.6)과 nvidia-docker 설치·검증까지 스크립트가 처리하도록 만들어, 누가 실행하든 같은 환경이 나오게 했습니다. 그 위에 이미지 생성 워크로드(Stable Diffusion WebUI · LoRA 학습용 kohya_ss · ComfyUI)를 표준 구성으로 올렸습니다. 사내 장비만으로 부족한 연산 자원은 클라우드로 확장했습니다. GCP에 NVIDIA A100 40GB 2장(a2-highgpu-2g) VM을 Terraform IaC로 구축하고, 리전·존별 A100 가용성 확인과 GPU quota 상향을 거친 뒤 앞선 마이그레이션에서 만든 Shared VPC 서브넷에 연결해 네트워크 표준을 유지했습니다. 연산 수요가 더 늘면서 외부 IDC에도 GPU 서버 20대를 구성했습니다. 앞서 사내 서버용으로 만든 세팅 스크립트를 그대로 재사용해, 20대를 한 대씩 맞추지 않고 같은 절차로 구축했습니다. 학습은 모델·배치가 카드 하나에 들어가지 않아 VRAM과 다중 GPU 연결(NVLink)이 필요하지만, 이미지 생성 추론은 요청마다 독립적이라 카드끼리 묶을 이유가 없고 대수에 비례해 처리량이 늘어납니다. 그래서 학습·대용량 작업은 GCP A100, 대량 추론은 IDC의 소비자용 RTX 40 시리즈 20대로 나눠 배치했습니다.

이후 신규 퍼블리싱 계약이 체결되면서 수요가 계속 이어져, 퇴사 시점까지 퍼블리셔 측과 협업하며 이 GPU 환경을 고도화했습니다.

성과

- Windows 수동 세팅을 Linux + 자동화 스크립트로 대체해 동일 환경 재현을 보장 — 이 스크립트가 이후 외부 IDC 20대 구축에 그대로 쓰이며, 1대용 절차가 GPU 플릿 전체의 표준 구축 절차가 됨
- GPU 서버를 손으로 구축하지 않고 Terraform 코드로 관리해 재구축·변경 추적을 표준화 — 기존 GCP IaC 저장소에 그대로 편입
- GPU 자원을 사내 서버 1대에서 사내 10대 · GCP A100 · 외부 IDC 20대 세 곳으로 확장 — 워크로드 특성(다중 GPU 연결이 필요한 학습 vs 요청 단위로 병렬화되는 추론)에 맞춰 카드 등급을 나눠 배치

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했습니다. 금융권·공공기관 프라이빗 클라우드 인프라를 설계·구축하고, 고객사 운영팀 교육을 직접 맡았습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 상이하여, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA(High Availability) 구성으로 설계하고, OpenStack으로 가상머신 관리 환경을 구축했습니다. Ceph로 블록·파일·오브젝트 스토리지를 통합 제공하고, 고객사 운영팀에 인프라 운영 교육을 병행했습니다.

성과

- 5개 고객사에 PaaS 솔루션 성공적으로 구축 및 납품
- Kubernetes 클러스터 HA 구성으로 안정적 운영
- Ansible 기반 서버 프로비저닝·구성 관리 자동화로 고객사별 배포 시간 단축, 환경 불일치(Configuration Drift) 0건 달성

프로젝트 2: DP사 Kubernetes 운영 교육 프로그램 설계·운영

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

DP사 운영팀이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족하여, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상되었습니다.

접근 전략

Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고, 실습 환경을 구성해 실무 중심으로 운영했습니다. OpenStack·Ceph 스토리지 운영 교육도 병행해 전체 PaaS 스택 운영 역량을 확보하도록 도왔습니다.

성과

- 수강자 평균 10명 · 회당 교육 4시간 규모로 커리큘럼을 운영하여 DP사 운영팀의 Kubernetes 자체 운영 체계 확립
- 교육 완료 후 고객사 기술 문의 건수 감소, 고객사가 장애를 스스로 대응할 수 있게 됨
- 교육 자료를 표준화하여 이후 타 고객사 교육에도 재활용

프로젝트 3: Kubernetes 인증서 자동 갱신 프로세스 개선

기여도 100% (단독 수행, 전체 고객사 배포로 확산) 2주 (2022.11)

문제

Kubernetes 클러스터 인증서가 1년마다 만료되어 고객사마다 연간 약 5시간의 갱신 작업이 필요했으며, 만료 시 서비스 장애가 발생할 위험이 있었습니다.

접근 전략

Kubernetes 인증서 관리 프로세스를 분석하고, kubeadm 설정을 수정하여 인증서 유효기간을 1년에서 10년으로 연장했습니다. 기존 운영 중인 클러스터에도 적용 가능한 자동화 스크립트를 개발하여 전체 고객사에 배포했습니다.

트러블슈팅

- kubeadm 설정 수정 방식은 kubeadm으로 초기화한 클러스터에만 적용 가능했고, v1.15.x / v1.16.x에서는 `kubeadm alpha certs renew` 의 알려진 버그로 일부 인증서가 갱신되지 않았습니다.
이런 버전에서는 kubeadm 명령에 의존하지 않고, `/etc/kubernetes` 백업 후 인증서 파일을 직접 10년으로 재생성하는 공개 오픈소스 스크립트 (`update-kube-cert`)를 도입했습니다. etcd·apiserver·controller-manager·scheduler·kubelet 재시작 절차와 묶어 적용해 버전에 관계없이 동작하도록 했습니다.

성과

- 인증서 갱신 주기 10배 연장 (연 1회 → 10년에 1회)
- 고객사 운영 부담 연간 약 50시간 절감 (고객사 10곳 기준)